

Pipeline NYC Taxi : Data Warehouse avec Snowflake & dbt

Non assigné



GS

Guillaume SOULAT créé : 10/02/26

Construire un pipeline de données professionnel en exploitant le dataset réel NYC Taxi (~40 millions de trajets, ~8 GB). Les apprenants conçoivent un Data Warehouse sur Snowflake avec une architecture en 3 couches (RAW → STAGING → FINAL), nettoient et transforment les données, puis produisent des analyses métier (KPIs financiers, opérationnels et géographiques). La partie avancée introduit dbt Core pour industrialiser les transformations SQL et GitHub Actions pour l'orchestration automatisée du pipeline.

Référentiels

Ressources

- [youtube](#)
- [Brief Complet](#)
- [Data WareHouse](#)
- [Snowflake](#)
- [Dbt](#)

Contexte du projet

La NYC Taxi & Limousine Commission publie chaque mois les données de millions de trajets en taxi jaune à New York, au format

Parquet. Ce dataset open data est une référence dans l'industrie pour s'exercer au traitement de données massives.

En tant que Data Engineer, vous êtes missionné pour construire le Data Warehouse analytique d'une société de transport.

Votre objectif : ingérer les données brutes, les nettoyer, les transformer et produire des tables d'analyse prêtes à

alimenter des dashboards décisionnels.

Le dataset présente des problèmes de qualité réels :

- 15,54% de valeurs manquantes
- 4,15% de montants négatifs

- 2,62% de trajets avec distance zéro
- Valeurs extrêmes aberrantes (distance > 1000 miles)

Ces anomalies reflètent les défis quotidiens du Data Engineer et justifient une approche rigoureuse de nettoyage et de validation.

L'architecture cible repose sur 3 schémas Snowflake :

- RAW : Données brutes importées sans modification depuis les fichiers Parquet
- STAGING : Données nettoyées, filtrées et enrichies (durée, vitesse, catégorisation)
- FINAL : Tables d'analyse métier (résumés quotidiens, analyses par zone, patterns horaires)

La partie avancée introduit dbt Core, l'outil standard de l'industrie pour versionner et tester les transformations SQL,

ainsi que GitHub Actions pour automatiser l'exécution mensuelle du pipeline.

Technologies utilisées : Snowflake, SQL, dbt Core, Python, GitHub Actions, Parquet.

Durée : 3 jours.

Modalités pédagogiques

Le projet est structuré en 2 parties progressives :

PARTIE 1 - TRONC COMMUN (obligatoire) :

Les apprenants travaillent individuellement ou en binôme sur la mise en place complète du Data Warehouse. Ils configurent

l'infrastructure Snowflake (warehouse, base, schémas), chargent les données via des stages externes, analysent les problèmes

de qualité et implémentent les transformations de nettoyage et d'enrichissement en SQL.

PARTIE 2 - OPTIONS AVANCÉES :

Les apprenants qui ont terminé le tronc commun choisissent parmi :

- Étape A : Industrialisation des transformations avec dbt Core (modèles staging/intermediate/marts, tests de qualité,

documentation auto-générée)

- Étape B : Orchestration automatisée avec GitHub Actions (déclenchement mensuel, secrets sécurisés, notifications)

Ressources pédagogiques fournies :

- Tutoriels vidéo (dbt + Snowflake, Airflow + NYC Taxi)
- Articles techniques (architecture ELT moderne, bonnes pratiques)
- Documentation officielle (Snowflake, NYC TLC Data Dictionary, dbt Core)
- Dictionnaire des données du dataset

Analyse réflexive :

À la fin du projet, chaque apprenant rédige une analyse réflexive couvrant :

- Les difficultés rencontrées et les solutions apportées
- Les choix techniques effectués et leur justification
- Les compétences acquises et celles à approfondir
- Le parallèle avec un contexte professionnel réel

Modalités d'évaluation

Évaluation en 2 volets : Tronc Commun (70% de la note) : - Soutenance orale de 10 minutes avec démonstration du Data Warehouse fonctionnel - Revue de code SQL et documentation - Vérification de l'architecture RAW/STAGING/FINAL - Qualité du nettoyage des données et pertinence des KPIs calculés Options Avancées (30% de la note) : - Démonstration des fonctionnalités avancées implémentées (dbt, orchestration, dashboard) - Qualité des tests de données et de la documentation auto-générée - Pertinence de l'automatisation mise en place L'analyse réflexive est évaluée sur la profondeur de l'introspection et la capacité à identifier les axes d'amélioration.

Livrables

Tronc Commun : 1. Architecture Snowflake : Base NYC_TAXI_DB avec les 3 schémas RAW, STAGING et FINAL créés et peuplés 2. Scripts SQL documentés : Requêtes de création de tables, chargement, nettoyage et transformation 3. Documentation README : Instructions d'exécution pas à pas, description de l'architecture et des choix techniques 4. Projet dbt Core : Modèles staging/intermediate/marts avec tests et documentation auto-générée 5. Pipeline orchestré : Workflow GitHub Actions avec déclenchement automatique et gestion des secrets 6. Dashboard : Visualisations interactives des KPIs principaux Transversal : 8. Repository GitHub : Code versionné avec historique de commits propre

Critères de performance

Architecture et ingestion : - Le Data Warehouse contient les 3 schémas RAW/STAGING/FINAL correctement structurés - Minimum 12 mois de données chargées et exploitables - Les données brutes sont stockées sans modification dans RAW Nettoyage et qualité : - Les montants négatifs, distances aberrantes et dates incohérentes sont filtrés - Les valeurs manquantes sont traitées de manière documentée - Le taux de rejet est quantifié et justifié Transformations : - Les

enrichissements sont calculés (durée, vitesse, taux de pourboire) - Les catégorisations sont implémentées (distance, période horaire, type de jour) - Minimum 3 tables d'analyse dans le schéma FINAL

Situation professionnelle

Optimisation de requêtes SQL complexes pour un data warehouse supportant des modèles IA

Besoin visé ou problème rencontré

Une entreprise utilise un data warehouse (PostgreSQL/Snowflake) contenant plusieurs téraoctets de données de transactions. Les data scientists lancent quotidiennement des requêtes SQL pour extraire les données d'entraînement de leurs modèles, mais ces requêtes prennent plusieurs heures à s'exécuter, bloquant les pipelines ML. L'entreprise souhaite optimiser drastiquement les performances des requêtes pour accélérer les cycles de développement.

6 compétences visées

Compétences visées

- C1. Automatiser l'extraction de données
 - 1. niveau 1, imiter
 - 2. niveau 2, adapter
 - 3. niveau 3, transposer
- C2. Développer des requêtes SQL
 - 1. niveau 1, imiter
 - 2. niveau 2, adapter
 - 3. niveau 3, transposer
- C6. Organiser et réaliser une veille
 - 1. niveau 1, imiter
 - 2. niveau 2, adapter
 - 3. niveau 3, transposer
- C9. Développer une API exposant un modèle d'IA
 - 1. niveau 1, imiter
 - 2. niveau 2, adapter
 - 3. niveau 3, transposer
- C15. Concevoir le cadre technique d'une application
 - 1. niveau 1, imiter

2. niveau 2, adapter

3. niveau 3, transposer

- C17. Développer les composants techniques et les interfaces d'une application

1. niveau 1, imiter

2. niveau 2, adapter

3. niveau 3, transposer

N

- r

a

f

f

i

c

h

e

r

q

u

e

l

e

s

3

p

r

e

m

i

è

r

e

s

c

o

m

p

é

t

e

n

c

e

s